

The Architectural Evolution to V5.2: A Briefing

This briefing outlines the engineering journey that led to **V5.2**, structuring the work as a formal software engineering experiment (Problem → Intervention → Measurement → Refinement).

1. The Baseline Problem: “Context Stuffing” (V4.5)

Where we started: We successfully built a pipeline that produced highly accurate architectural documents. **The Problem:** The architecture was not scalable. We were forcing the entire OptArrow repository directly into the LLM’s context window. We quickly hit the 16,384 token limit, which caused the LLM to hallucinate, crash into infinite escaping loops, and made the system computationally unviable for larger codebases. We needed to decouple the LLM token cost from the repository size.

2. Experiment 1: Bounded Lexical Retrieval (V5.0)

The Intervention: We introduced a RAG (Retrieval-Augmented Generation) architecture. We implemented a “Context Budget Manager” to strictly cap the LLM at 6,000 tokens, and we used a lexical retriever to fetch only the files that matched the ISO standard’s terminology. **The Result (The Trade-off):** We successfully proved we could cap the tokens, but we suffered catastrophic **architectural information loss**. * **Why it failed:** 1. We were crudely chopping files at 2,000 characters, meaning deep algorithms (like Julia solvers) were thrown away before the LLM ever saw them. 2. Because ISO standards use generic words like “system” or “process”, our naive retriever ranked generic CI/CD deployment files higher than the actual core Python runtime code. 3. The agent responsible for mapping the global repository blindly grabbed the first 25 files in alphabetical order, completely blinding it to the rest of the system.

3. Experiment 2: Granular, Relevance-Aware Retrieval (V5.1)

The Intervention: To recover the lost architectural depth, we stopped randomly chopping files and restored **AST (Abstract Syntax Tree) extraction** to perfectly slice out specific Python classes and functions. We also added “stopword filtering” to ignore generic terms, preventing CI/CD files from hijacking the context limit. **The Result:** Architectural specificity (like gRPC and FastAPI) returned to the generated document. However, three critical bottlenecks remained: 1. **Language Asymmetry:** The AST extractor only worked for Python. Julia files were still being crudely chopped, skewing the architecture heavily toward Python. 2. **Alphabetical Bias:** The global mapping agent was still grabbing the first 15,000 tokens it could find, meaning if critical files were located at the bottom of the directory tree, they were ignored. 3. **Semantic Laundering (Hallucination Risk):** The final compiling agent was given a list of facts and told “*this came from EV-42*”, but it couldn’t actually read EV-42. It had to blindly trust the upstream agents, creating a massive vulnerability for hallucinations.

4. The Ultimate Architecture (V5.2)

The Final Interventions: V5.2 resolved all the remaining constraints, achieving peak scalability and epistemic rigor. 1. **Language Parity (Julia AST):** We built a robust parser that natively understands Julia (function, struct, module) so that OptArrow’s dual-language architecture is treated with equal granularity. 2. **Representation-Aware Sampling:** We eliminated the alphabetical bias. Instead of grabbing the first 15,000 tokens, the global agent is now fed a mathematically balanced diet of the codebase: **20% diagrams, 35% Python, 35% Julia, and 10% configuration**. This guarantees a 360-degree view of the system. 3. **Strict Provenance Verification:** We completely eliminated the “Semantic Laundering” vulnerability. When the compiling agent is asked to write the final document, the pipeline intercepts the `evidence_ids`, retrieves the exact raw code snippets, and attaches them to the prompt. The agent physically verifies the code before writing the claim.

Conclusion

V5.2 successfully proves that context reduction alone is insufficient for automated compliance. By pairing strict token budgets with AST-level granularity, mathematically balanced repository sampling, and end-to-end provenance verification, we built a system that scales to any repository size without sacrificing an ounce of technical specificity or epistemic truth.